HackPSU Implementation Plan — Parallel Workstreams

Team: 3 members working in parallel **Total Time:** 24 hours **Goal:** Working demo with document upload → extraction → lookup → appeal letter generation across 4 insurance types (ACA, Medicare, ERISA, Medicaid)

COMPLETE CARC CODE COVERAGE

These are all the codes our JSON database will handle, organized by denial category. Each code maps to one category, and each category has 4 appeal tracks (ACA, Medicare, ERISA, Medicaid).

Category 1: Prior Authorization

Code	Official Description	Consumer Meaning
CO-197	Precertification/authorization/notification absent	You or your doctor didn't get pre-approval before the service
CO-15	The authorization number is missing, invalid, or does not apply to the billed services or provider	Auth number wasn't included or doesn't match
CO-27	Expenses incurred after coverage terminated	Sometimes used when auth expired before service was rendered

Appeal angle: Authorization is the provider's responsibility, not the patient's. Retroactive auth is available. Treatment was medically necessary.

Category 2: Medical Necessity

Code	Official Description	Consumer Meaning
CO-50	These are non-covered services because this is not deemed a "medical necessity" by the payer	Insurer says you didn't need this treatment
CO-150	Payer deems the information submitted does not support this level of service	Insurer says the documentation doesn't justify this level of care
CO-151	Payer deems the information submitted does not support this many/frequency of services	Insurer says you got this service too many times
CO-54	Payer deems the information submitted does not support this level of service, this many services, this length of service, this dosage, or this day's supply	Catch-all for "we don't think you needed this much"

Appeal angle: Request peer-to-peer review. Cite treating physician's clinical judgment. Provide additional medical records and clinical notes.

Category 3: Out-of-Network / Coordination of Benefits

Code	Official Description	Consumer Meaning
CO-11	The diagnosis is inconsistent with the procedure	Often used when out-of-network billing has code mismatches
CO-22	This care may be covered by another payer per coordination of benefits	You have multiple insurance and they're pointing at each other
CO-109	Claim/service not covered by this payer/contractor	This specific insurer says it's not their responsibility

Appeal angle: Verify correct billing order for COB. For out-of-network, cite No Surprises Act protections (if emergency or certain other situations). Clarify primary vs secondary payer.

Category 4: Coding / Billing Error

Code	Official Description	Consumer Meaning
CO-4	The procedure code is inconsistent with the modifier used or a required modifier is missing	Coding mistake — wrong modifier on the claim
CO-16	Claim/service lacks information or has submission/billing error(s)	Something was missing or wrong on the claim form
CO-97	The benefit for this service is included in the payment/allowance for another service/procedure	This was bundled into another charge — duplicate or overlap
CO-18	Exact duplicate claim/service	This exact claim was already submitted and processed
CO-45	Charge exceeds fee schedule/maximum allowable or contracted/legislated fee arrangement	Billed amount is higher than what the insurer will pay

Appeal angle: Contact provider's billing department to correct and resubmit. Patient should not be balance-billed for coding errors. If CO-45, verify the contracted rate is correct.

Category 5: Timely Filing

Code	Official Description	Consumer Meaning
CO-29	The time limit for filing has expired	The claim was submitted too late

Appeal angle: Limited options. Check if there were extenuating circumstances (payer delays, retroactive eligibility). Some states have exceptions. Verify the filing deadline — it varies by payer.

Category 6: Experimental / Investigational

Code	Official Description	Consumer Meaning
CO-55	Procedure/treatment/drug is deemed experimental/ investigational by the payer	Insurer says this treatment is experimental
CO-56	Procedure/treatment has not been deemed "proven to be effective" by the payer	Insurer says there's not enough evidence this works

Appeal angle: Provide peer-reviewed studies and clinical evidence. Request peer-to-peer review. Cite FDA approval status if applicable. Some state laws mandate coverage of certain "experimental" treatments (e.g., cancer clinical trials).

Category 7: Coverage / Eligibility

Code	Official Description	Consumer Meaning
CO-26	Expenses incurred prior to coverage	Service happened before your insurance started
CO-96	Non-covered charge(s)	Your plan simply doesn't cover this service

Appeal angle: Verify eligibility dates are correct. For CO-96, check if the service should be covered under preventive care mandates or essential health benefits (ACA). Request the specific plan exclusion language.

TOTAL: 7 categories, 20 CARC codes, 4 plan types = 28 JSON track entries + 50 state contact entries for Medicaid

---

TEAM MEMBER 1: DATA & DATABASE

Role: Build the complete JSON database, Medicaid state contact directory, and synthetic demo documents **Deliverable:** `denial_database.json` + `state_medicaid_contacts.json` + 4 synthetic denial letter PDFs

Hour-by-Hour Plan

Hours 1-2: Setup & Code Research

- Read through the CARC code table above and verify descriptions against X12's official list at x12.org/codes/claim-adjustment-reason-codes
- Set up the JSON file skeleton with all 7 categories
- For each category, confirm the CARC codes are correctly assigned

Hours 2-6: Generate JSON Entries Using LLM

Use Claude or GPT-4 to generate entries. For each of the 7 categories, prompt the LLM with:

```
Generate a JSON entry for our insurance denial appeal database.

Category: [e.g., "Prior Authorization"]
CARC codes in this category: [e.g., "CO-197, CO-15, CO-27"]

For each of the following 4 insurance types, provide:
1. The specific federal regulations that govern appeals for this denial type
2. The appeal deadline
3. The step-by-step appeal process
4. The specific appeal strategy and arguments
5. What evidence the patient should include
6. The escalation path if the appeal is denied

Insurance types:
- ACA Marketplace (governed by 42 USC §300gg-19, 29 CFR §2590.715-2719)
- Medicare (governed by 42 CFR Parts 405 and 422)
- Employer-Sponsored / ERISA (governed by 29 USC §1133, 29 CFR §2560.503-1)
- Medicaid — FEDERAL FLOOR ONLY (governed by 42 CFR Part 431 Subpart E, §431.200-
§431.246). Do NOT include state-specific rules. Focus on the federally guaranteed
right to a fair hearing.

Output as a JSON object matching this schema:
{
  "denial_category": "string",
  "category_id": "string (snake_case)",
  "carc_codes": ["string"],
  "description": "string — plain English description",
  "tracks": {
    "aca": {
      "regulations": [{"citation": "string", "summary": "string"}],
      "appeal_deadline": "string",
      "appeal_process": ["step 1", "step 2", ...],
      "appeal_strategy": "string",
      "template_key": "string",
      "escalation": "string",
      "required_evidence": ["string"]
    },
    "medicare": { ... same structure ... },
    "erisa": { ... same structure ... },
    "medicaid": { ... same structure, federal floor only ... }
  },
  "common_grounds": "string — general appeal arguments for this category"
}
```

Run this prompt 7 times (once per category). Save all outputs into `denial_database.json` as an array.

Hours 6-8: Build Medicaid State Contact Directory

Generate `state_medicaid_contacts.json` using an LLM. Prompt:

```
For all 50 US states plus DC, provide the following Medicaid
appeal contact information in JSON format:

{
  "PA": {
    "state_name": "Pennsylvania",
    "agency": "PA Department of Human Services",
    "appeal_body": "Bureau of Hearings and Appeals",
    "phone": "1-800-XXX-XXXX",
    "online_portal": "https://...",
    "notes": "Any state-specific appeal quirks (e.g., must appeal to MCO first)"
  },
  ...
}
```

This data is used to personalize the Medicaid appeal letter with the correct state agency name, contact info, and mailing address. The legal citations in the letter remain federal (42 CFR Part 431 Subpart E) — the state info is only for routing.

Important: Spot-check at least 5-10 states (PA, NY, CA, TX, FL, IL, OH) by Googling "[state] Medicaid fair hearing" and confirming the agency name and phone number. The LLM is generally good at agency names but phone numbers can be wrong.

Hours 8-12: Build Synthetic Demo Documents

Create 4 fake denial letters as PDFs (use any PDF tool or even Google Docs → export). Each should look like a realistic denial letter with:

Demo Doc 1 — ACA Prior Auth Denial:

- Fake insurer: "Keystone Health Plan"
- Fake patient: "Jane Smith"
- Denial code: CO-197
- Procedure: CPT 27447 (knee replacement)
- Denial reason: "Prior authorization was not obtained for this procedure"
- Date of service, claim number, provider name all fake

Demo Doc 2 — Medicare Medical Necessity Denial:

- Fake insurer: "Medicare Part B — Novitas Solutions"
- Fake patient: "Robert Johnson"
- Denial code: CO-50
- Procedure: CPT 99214 (office visit, level 4)
- Denial reason: "The submitted information does not support medical necessity"
- Include a fake Medicare Summary Notice (MSN) format

Demo Doc 3 — ERISA Coding Error:

- Fake insurer: "Anthem Blue Cross (via employer plan)"
- Fake patient: "Maria Garcia"
- Denial code: CO-16
- Procedure: CPT 99213 (office visit, level 3)
- Denial reason: "Claim lacks required information — referring provider NPI missing"

Demo Doc 4 — Medicaid Coverage Denial:

- Fake insurer: "PA Health & Wellness (Medicaid MCO)"
- Fake patient: "David Williams"
- Denial code: CO-96
- Procedure: CPT 97110 (physical therapy)
- Denial reason: "Non-covered charge — service exceeds benefit limit"
- Include a state-specific MCO letterhead to make it realistic

Hours 12-14: Build CARC Code Lookup Index

Create a separate `carc_lookup.json` that maps every individual code to its category:

```json
{
  "197": "prior_authorization",
  "15": "prior_authorization",
  "27": "prior_authorization",
  "50": "medical_necessity",
  "150": "medical_necessity",
  "151": "medical_necessity",
  "54": "medical_necessity",
  "11": "out_of_network",
  "22": "out_of_network",
  "109": "out_of_network",
  "4": "coding_error",
  "16": "coding_error",
  "97": "coding_error",
  "18": "coding_error",
  "45": "coding_error",
  "29": "timely_filing",
  "55": "experimental",
  "56": "experimental",
  "26": "coverage_eligibility",
  "96": "coverage_eligibility"
}
```

This is the fast-path lookup. Backend receives extracted code → looks up category → pulls the full entry from `denial_database.json`.

Hours 14-16: Category Description Index (for prose classification fallback)

Create `category_descriptions.json` — used when no CARC code is found and the LLM needs to classify a prose denial reason:

```json
[
  {
    "category_id": "prior_authorization",
    "description": "The claim was denied because the provider did not obtain prior a
    "keywords": ["prior authorization", "precertification", "pre-approval", "authori
  },
  {
    "category_id": "medical_necessity",
    "description": "The claim was denied because the payer determined the service wa
    "keywords": ["not medically necessary", "medical necessity", "level of service",
  },
  ...
]
```

Hours 16-20: Integration Testing with Backend

Work with Member 2 to test:

- Does the backend correctly look up codes from `carc_lookup.json`?
- Does the backend correctly pull full entries from `denial_database.json` for all 4 tracks?
- Does the Medicaid flow correctly pull state contact info from `state_medicaid_contacts.json`?
- Does the prose classification fallback work with `category_descriptions.json`?
- Do all 4 synthetic demo docs parse correctly?

Hours 20-24: Demo Prep

- Practice the demo flow with all 4 synthetic documents
- Prepare talking points about the database methodology
- Have the "honest answer" ready for judges: "LLM-generated for prototype, would be legally verified for production. Medicaid uses federal floor citations with state-specific routing."

TEAM MEMBER 2: BACKEND & AI PIPELINE

Role: Build the FastAPI backend with the 3-stage pipeline **Deliverable:** Working API at `/api/analyze` that accepts a document or form data and returns a structured appeal

Hour-by-Hour Plan

Hours 1-2: Project Setup

```bash
mkdir appeal-assistant && cd appeal-assistant
python -m venv venv && source venv/bin/activate
pip install fastapi uvicorn python-multipart google-generativeai PyPDF2 Pillow pytes
```

Set up the FastAPI project structure:

```
backend/
├── main.py              # FastAPI app, routes
├── pipeline/
│   ├── extractor.py     # Stage 1: Document → structured fields
│   ├── classifier.py    # Stage 2: Fields → denial category + track
│   ├── generator.py     # Stage 3: Category + track → appeal letter
│   └── guardrails.py    # Confidence checks and refusal logic
├── data/
│   ├── denial_database.json    # (from Member 1)
│   ├── carc_lookup.json        # (from Member 1)
│   └── category_descriptions.json  # (from Member 1)
└── prompts/
    ├── extraction_prompt.txt
    ├── classification_prompt.txt
    └── generation_prompt.txt
```

Hours 2-4: Stage 1 — Extraction (`extractor.py`)

Build the document ingestion and extraction pipeline:

```python
# Pseudocode for extractor.py

async def extract_from_document(file_bytes, file_type):
    """Extract structured denial info from uploaded document."""

    # 1. Convert document to text
    if file_type == "pdf":
        text = extract_text_from_pdf(file_bytes)  # PyPDF2
    elif file_type in ["png", "jpg", "jpeg"]:
        text = ocr_image(file_bytes)  # pytesseract
    else:
        raise ValueError("Unsupported file type")

    # 2. Send to LLM for structured extraction
    extraction_prompt = load_prompt("extraction_prompt.txt")

    response = call_gemini(
        prompt=extraction_prompt.format(document_text=text),
        temperature=0  # CRITICAL: deterministic output
    )

    # 3. Parse LLM response as JSON
    extracted = parse_json_response(response)

    # 4. Return structured fields
    return {
        "carc_code": extracted.get("carc_code"),      # e.g., "197" or null
        "procedure_code": extracted.get("procedure_code"),  # e.g., "27447"
        "denial_reason_prose": extracted.get("denial_reason"),
        "insurer_name": extracted.get("insurer_name"),
        "patient_name": extracted.get("patient_name"),
        "date_of_service": extracted.get("date_of_service"),
        "date_of_denial": extracted.get("date_of_denial"),
        "claim_number": extracted.get("claim_number"),
        "plan_type": extracted.get("plan_type"),  # "aca", "medicare", "erisa", or n
        "confidence": extracted.get("confidence")  # "high", "medium", "low"
    }
```

Extraction prompt (`prompts/extraction_prompt.txt`):

```
You are a medical billing document parser. Extract the following
fields from this denial letter or Explanation of Benefits.

Return ONLY a JSON object with these fields. If a field is not
found, set it to null. Do not guess or infer — only extract what
is explicitly stated in the document.

Fields to extract:
- carc_code: The CARC denial reason code (numeric only, e.g., "197", "50"). Look
for patterns like "CO-197", "CARC 197", "Reason Code 197", or just "197" near
denial reason text.
- procedure_code: The CPT or HCPCS code for the denied service (e.g., "27447",
"99213")
- denial_reason: The plain-English reason for denial as stated in the document
- insurer_name: The insurance company name
- patient_name: The patient's name
- date_of_service: The date of the denied service
- date_of_denial: The date of the denial letter/EOB
- claim_number: The claim or reference number
- plan_type: One of "aca", "medicare", "erisa", or null. Clues: Medicare logos/
language → "medicare". Employer plan references → "erisa". Marketplace/
Healthcare.gov references → "aca".
- confidence: Your confidence in the extraction — "high" if all key fields
(especially carc_code or denial_reason) were clearly found, "medium" if some were
ambiguous, "low" if the document was hard to parse.

DOCUMENT TEXT:
{document_text}
```

Hours 4-6: Stage 2 — Classification (`classifier.py`)

python

```python
# Pseudocode for classifier.py

import json

# Load databases (on startup)
carc_lookup = json.load(open("data/carc_lookup.json"))
denial_db = json.load(open("data/denial_database.json"))
category_descriptions = json.load(open("data/category_descriptions.json"))

async def classify_denial(extracted_fields):
    """Map extracted fields to a denial category and plan track."""

    carc_code = extracted_fields.get("carc_code")
    plan_type = extracted_fields.get("plan_type")

    # FAST PATH: Direct code lookup
    if carc_code and carc_code in carc_lookup:
        category_id = carc_lookup[carc_code]
        confidence = "high"

    # FALLBACK: Prose classification
    elif extracted_fields.get("denial_reason_prose"):
        category_id, confidence = await classify_by_prose(
            extracted_fields["denial_reason_prose"]
        )

    # REFUSAL: Can't identify denial type
    else:
        return {
            "status": "insufficient_data",
            "message": "We couldn't identify the basis for this denial.",
            "suggestion": "Try entering your denial code manually, or contact your s
        }

    # GUARDRAIL: Check confidence threshold
    if confidence == "low":
        return {
            "status": "low_confidence",
            "message": f"We think this might be a {category_id} denial, but we're no
            "suggestion": "Please verify: does this look right?",
            "tentative_category": category_id
        }

    # Pull full database entry
    db_entry = next(e for e in denial_db if e["category_id"] == category_id)
```

```
    db_entry = next(e for e in denial_db if e["category_id"] == category_id)

    # Determine plan track
    if not plan_type:
        return {
            "status": "need_plan_type",
            "category": db_entry,
            "message": "We identified your denial type but need to know your insuran
        }

    track = db_entry["tracks"].get(plan_type)

    return {
        "status": "success",
        "category_id": category_id,
        "category": db_entry,
        "track": track,
        "plan_type": plan_type,
        "extracted_fields": extracted_fields
    }


async def classify_by_prose(denial_reason_text):
    """Use LLM to classify a prose denial reason into a category."""

    categories_text = json.dumps(category_descriptions, indent=2)

    prompt = f"""Given this denial reason from an insurance document:

"{denial_reason_text}"

Which of the following denial categories does it best match?

{categories_text}

Return ONLY a JSON object with:
- "category_id": the matching category_id
- "confidence": "high" if it clearly matches, "medium" if it's a reasonable match, "

If none of the categories match well, return:
- "category_id": null
- "confidence": "low"
"""

    response = call_gemini(prompt=prompt, temperature=0)
    result = parse_json_response(response)
    return result.get("category_id"), result.get("confidence", "low")
```

Hours 6-10: Stage 3 — Appeal Letter Generation (`generator.py`)

python

```python
# Pseudocode for generator.py

async def generate_appeal_letter(classification_result):
    """Generate appeal letter from classified denial + database entry."""

    extracted = classification_result["extracted_fields"]
    track = classification_result["track"]
    category = classification_result["category"]
    plan_type = classification_result["plan_type"]

    # Build the constrained prompt
    regulations_list = "\n".join([
        f"- {r['citation']}: {r['summary']}"
        for r in track["regulations"]
    ])

    evidence_list = "\n".join([f"- {e}" for e in track["required_evidence"]])

    appeal_process = "\n".join([
        f"  {i+1}. {step}"
        for i, step in enumerate(track["appeal_process"])
    ])

    prompt = f"""You are drafting a health insurance appeal letter.
Use ONLY the regulatory citations and appeal grounds provided below.
Do not invent or infer any legal citations. If information is missing,
use [PLACEHOLDER] brackets.

DENIAL DETAILS:
- Patient: {extracted.get("patient_name") or "[PATIENT NAME]"}
- Insurance: {extracted.get("insurer_name") or "[INSURANCE COMPANY]"}
- Claim Number: {extracted.get("claim_number") or "[CLAIM NUMBER]"}
- Date of Service: {extracted.get("date_of_service") or "[DATE OF SERVICE]"}
- Date of Denial: {extracted.get("date_of_denial") or "[DATE OF DENIAL]"}
- Procedure: {extracted.get("procedure_code") or "[PROCEDURE CODE]"}
- Denial Code: {extracted.get("carc_code") or "N/A"}
- Denial Reason: {extracted.get("denial_reason_prose") or category["description"]}

INSURANCE TYPE: {plan_type.upper()}
APPEAL DEADLINE: {track["appeal_deadline"]}

REQUIRED REGULATORY CITATIONS (use these verbatim):
{regulations_list}

APPEAL STRATEGY:
```

```
APPEAL STRATEGY:
{track["appeal_strategy"]}

APPEAL GROUNDS:
{category["common_grounds"]}

REQUIRED EVIDENCE TO REFERENCE:
{evidence_list}

ESCALATION IF DENIED:
{track["escalation"]}

LETTER STRUCTURE:
1. Opening: Formal appeal statement with claim number and denial date
2. Denial summary: What was denied and the stated reason
3. Legal basis: Cite ONLY the regulatory provisions listed above
4. Argument: Why the denial should be overturned using the appeal
   grounds above
5. Evidence: Reference supporting documents the patient should attach
6. Deadline notice: State the insurer's response deadline per regulations
7. Escalation warning: Note the next step if this appeal is denied
8. Closing: Request written decision, provide contact info

Write in a firm but professional tone. Address to the insurer's
appeals department. Output the complete letter only — no preamble
or explanation."""

    response = call_gemini(prompt=prompt, temperature=0)
    return response
```

Hours 10-12: API Routes (`main.py`)

python

```python
# Pseudocode for main.py

from fastapi import FastAPI, UploadFile, File, Form
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)

@app.post("/api/analyze/document")
async def analyze_document(
    file: UploadFile = File(...),
    plan_type: str = Form(None)  # optional, can be provided or extracted
):
    """Primary path: Document upload."""

    # Stage 1: Extract
    file_bytes = await file.read()
    file_type = file.filename.split(".")[-1].lower()
    extracted = await extract_from_document(file_bytes, file_type)

    # Override plan_type if user provided it
    if plan_type:
        extracted["plan_type"] = plan_type

    # Stage 2: Classify
    classification = await classify_denial(extracted)

    if classification["status"] != "success":
        return classification  # Frontend handles the error/follow-up

    # Stage 3: Generate
    appeal_letter = await generate_appeal_letter(classification)

    return {
        "status": "success",
        "extracted_fields": extracted,
        "denial_category": classification["category_id"],
        "track_info": classification["track"],
        "appeal_letter": appeal_letter
    }


@app.post("/api/analyze/manual")
async def analyze_manual(
    denial_code: str = Form(None),
```

```
    carc_code: str = Form(None),
    denial_reason: str = Form(None),
    plan_type: str = Form(...),
    patient_name: str = Form(None),
    insurer_name: str = Form(None),
    claim_number: str = Form(None),
    date_of_service: str = Form(None),
    date_of_denial: str = Form(None),
    procedure_code: str = Form(None)
):
    """Fallback path: Guided manual input."""

    extracted = {
        "carc_code": carc_code,
        "denial_reason_prose": denial_reason,
        "plan_type": plan_type,
        "patient_name": patient_name,
        "insurer_name": insurer_name,
        "claim_number": claim_number,
        "date_of_service": date_of_service,
        "date_of_denial": date_of_denial,
        "procedure_code": procedure_code,
        "confidence": "high"  # User provided the data
    }

    classification = await classify_denial(extracted)

    if classification["status"] != "success":
        return classification

    appeal_letter = await generate_appeal_letter(classification)

    return {
        "status": "success",
        "extracted_fields": extracted,
        "denial_category": classification["category_id"],
        "track_info": classification["track"],
        "appeal_letter": appeal_letter
    }


@app.get("/api/categories")
async def list_categories():
    """Return all denial categories for the frontend dropdown."""
    return [
        {"id": e["category_id"], "name": e["denial_category"], "codes": e["carc_code
        for e in denial_db
```

```
    ]
```

Hours 12-14: Guardrails (`guardrails.py`)

```python
# Confidence threshold refusal
def check_extraction_confidence(extracted_fields):
    """Return False if we should refuse to proceed."""
    if extracted_fields.get("confidence") == "low":
        return False
    if not extracted_fields.get("carc_code") and not extracted_fields.get("denial_re
        return False
    return True

# Post-generation citation check (simplified version)
def validate_citations_in_letter(letter_text, track_regulations):
    """Check that citations in the letter exist in our database."""
    known_citations = [r["citation"] for r in track_regulations]
    warnings = []

    # Simple regex check for citation patterns
    import re
    found_citations = re.findall(r'\d+ (?:USC|CFR) §[\d.]+(?:\([a-z]\))?', letter_te

    for citation in found_citations:
        if not any(citation in known for known in known_citations):
            warnings.append(f"Unverified citation found: {citation}")

    return warnings
```

Hours 14-16: Gemini API Helper

```python
# gemini_client.py

import google.generativeai as genai
import json

genai.configure(api_key="YOUR_API_KEY")  # Use env variable in real code

def call_gemini(prompt, temperature=0):
    model = genai.GenerativeModel("gemini-1.5-flash")
    response = model.generate_content(
        prompt,
        generation_config=genai.GenerationConfig(temperature=temperature)
    )
    return response.text

def parse_json_response(text):
    """Safely parse JSON from LLM response, stripping markdown fences."""
    cleaned = text.strip()
    if cleaned.startswith("```json"):
        cleaned = cleaned[7:]
    if cleaned.startswith("```"):
        cleaned = cleaned[3:]
    if cleaned.endswith("```"):
        cleaned = cleaned[:-3]
    return json.loads(cleaned.strip())
```

Hours 16-20: Integration Testing

- Test with Member 1's synthetic demo documents
- Test the manual input fallback path
- Test the refusal paths (bad document, no code found, low confidence)
- Test all 3 plan types produce different appeal letters
- Test edge cases: missing fields, unsupported file types

Hours 20-22: Connect to Frontend

- Work with Member 3 to ensure API responses match what the frontend expects
- Handle CORS issues
- Test end-to-end flow

Hours 22-24: Bug Fixes & Demo Prep

- Fix any issues found during testing
- Ensure the demo documents produce clean results every time
- Have the manual input path ready as backup

TEAM MEMBER 3: FRONTEND

Role: Build the React + Tailwind UI **Deliverable:** Working frontend with document upload, extraction review, and appeal letter display

Hour-by-Hour Plan

Hours 1-2: Project Setup

```bash
npx create-react-app appeal-assistant-ui
cd appeal-assistant-ui
npm install axios react-dropzone react-markdown
```

Or use Vite:

```bash
npm create vite@latest appeal-assistant-ui -- --template react
cd appeal-assistant-ui
npm install axios react-dropzone react-markdown
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Hours 2-6: Core Layout & Components

Build 4 main screens/states:

```
1. UPLOAD SCREEN     → User uploads document or clicks "Enter manually"
2. REVIEW SCREEN     → Show extracted fields, user confirms or corrects
3. PLAN TYPE SCREEN  → Ask insurance type if not auto-detected
4. RESULTS SCREEN    → Side-by-side: denial breakdown | appeal letter
```

Component structure:

```
src/
├── App.jsx
├── components/
│   ├── UploadScreen.jsx      # Drag-and-drop upload + manual entry toggle
│   ├── ManualEntryForm.jsx   # Guided form (fallback path)
│   ├── ReviewScreen.jsx      # Show extracted fields for confirmation
│   ├── PlanTypeSelector.jsx  # Insurance type question
│   ├── ResultsScreen.jsx     # Denial breakdown + appeal letter
│   ├── AppealLetter.jsx      # Formatted appeal letter with copy button
│   ├── DenialBreakdown.jsx   # Parsed denial info display
│   ├── ErrorScreen.jsx       # "We couldn't identify..." refusal screen
│   └── LoadingSpinner.jsx    # Loading state during API calls
├── api/
│   └── client.js             # Axios wrapper for backend calls
└── styles/
    └── index.css             # Tailwind imports
```

Hours 6-10: Upload Screen (`UploadScreen.jsx`)

Key features:

- Drag-and-drop zone using `react-dropzone`
- Accepts PDF, PNG, JPG
- Big clear call-to-action: "Upload your denial letter or EOB"
- Below the upload zone: "Don't have your document? Enter your info manually →" link
- Insurance type selector (optional at this stage): Employer / ACA Marketplace / Medicare / Medicaid
- If Medicaid selected → show helpful redirect screen with state resources, don't proceed to analysis
- Upload button triggers POST to `/api/analyze/document`

Hours 10-14: Review Screen (`ReviewScreen.jsx`)

After extraction, display the extracted fields in editable form:

```
┌──────────────────────────────────────────┐
│  We found the following from your document: │
│                                          │
│  Denial Code:     [CO-197       ] ✏️ edit     │
│  Procedure:       [CPT 27447    ] ✏️ edit     │
│  Insurer:         [Keystone HP ] ✏️ edit     │
│  Denial Reason:   [Prior auth not obtained]  │
│  Date of Service:[01/15/2026   ] ✏️ edit     │
│  Claim Number:    [CLM-98765    ] ✏️ edit     │
│  Insurance Type: [ACA ▼        ]             │
│                                          │
│  [ Does this look right? → Generate Appeal ]│
│  [ Something's wrong → Edit fields ]         │
└──────────────────────────────────────────┘
```

- All fields editable
- Insurance type dropdown if not auto-detected
- "Generate Appeal" button triggers Stage 2 + 3 on backend
- Show confidence indicator (green/yellow/red) based on extraction confidence

Hours 14-18: Results Screen (`ResultsScreen.jsx`)

Side-by-side layout:

```
┌──────────────────────┬──────────────────────────┐
│   DENIAL BREAKDOWN   │     YOUR APPEAL LETTER   │
│                      │                          │
│ Category:            │ [Full formatted letter]  │
│   Prior Authorization│                          │
│                      │                          │
│ Your Insurance:      │                          │
│   ACA Marketplace    │                          │
│                      │                          │
│ Appeal Deadline:     │                          │
│   30 days from denial│                          │
│                      │                          │
│ Regulations:         │                          │
│   42 USC §300gg-19(b)│                          │
│   29 CFR §2590...    │                          │
│                      │                          │
│ Next Steps:          │                          │
│   1. Gather evidence │                          │
│   2. Send this letter│                          │
│   3. If denied, file │                          │
│      external review │                          │
│                      │  [ Copy Letter ] [ ↓ ]   │
│ ⚠️ This is not legal │  [ Download PDF ]        │
│ advice. Consult a    │                          │
│ lawyer for complex   │                          │
│ cases.               │                          │
└──────────────────────┴──────────────────────────┘
```

Key features:

- "Copy Letter" button → copies appeal text to clipboard
- "Download PDF" → nice-to-have if time permits
- Required evidence checklist with checkboxes
- Clear disclaimer at the bottom
- "Start Over" button to return to upload screen

Hours 18-20: Manual Entry Form (`ManualEntryForm.jsx`)

Fallback path form:

1. Insurance type (required): radio buttons for Employer / ACA / Medicare / Medicaid
2. Denial code (optional): text input with help text "Look for CO-XXX on your EOB"
3. OR denial reason (if no code): dropdown of the 7 categories in plain English
4. Patient name, insurer, claim number, dates (all optional but helpful)
5. Submit → POST to `/api/analyze/manual`

Hours 20-22: Error Handling & Edge Cases

- `ErrorScreen.jsx`: Friendly error when system can't identify the denial
  - "We couldn't confidently identify the basis for your denial"
  - Link to state insurance commissioner
  - Option to try manual entry instead
- Loading states with progress messages during LLM calls (these can take 5-10 seconds)
- Handle API timeouts gracefully
- Handle case where backend returns `need_plan_type` status

Hours 22-24: Polish & Demo Prep

- Responsive layout (demo might be on a projector)
- Make sure copy-to-clipboard works
- Test full flow with all 3 demo documents
- Test manual entry fallback
- Test error states
- Clean up any console errors

INTEGRATION TIMELINE

Hour	Member 1 (Data)	Member 2 (Backend)	Member 3 (Frontend)
1-2	CARC research, JSON skeleton	Project setup, FastAPI scaffold	Project setup, React scaffold
2-6	LLM-generate all 7 category entries (4 tracks each)	Stage 1: Extraction pipeline	Upload screen + layout
6-8	Build Medicaid state contact directory (50 states)	Stage 2: Classification	Review screen
8-12	Build 4 synthetic demo documents	Stage 3: Generation	Results screen (side-by-side)
12-14	Build lookup indexes	API routes + guardrails	Manual entry form
14-16	Spot-check state contacts (PA, NY, CA, TX, FL)	Gemini helper + Medicaid state lookup logic	State dropdown for Medicaid
16-20	🔗 **INTEGRATION: Test data → backend**	🔗 **INTEGRATION: Test backend → frontend**	🔗 **INTEGRATION: Test frontend → backend**
20-22	Final data fixes	Connect to frontend, fix bugs	Error handling, polish
22-24	**Demo prep (all hands)**	**Demo prep (all hands)**	**Demo prep (all hands)**

Critical Handoff Points

- **Hour 6:** Member 1 delivers first draft of `denial_database.json` (with all 4 tracks) to Member 2 for testing
- **Hour 8:** Member 1 delivers `state_medicaid_contacts.json` to Member 2
- **Hour 10:** Member 1 delivers `carc_lookup.json` and `category_descriptions.json`
- **Hour 12:** Member 2 has working API endpoints for Member 3 to connect to
- **Hour 14:** Member 1 delivers all 4 synthetic demo PDFs for end-to-end testing
- **Hour 16:** All three begin integration testing together

API CONTRACT (So frontend and backend can develop in parallel)

POST `/api/analyze/document`

Request: multipart form — `file` (PDF/image) + optional `plan_type` string + optional `state` string (required for Medicaid)

Response (success):

```json
{
  "status": "success",
  "extracted_fields": {
    "carc_code": "197",
    "procedure_code": "27447",
    "denial_reason_prose": "Prior authorization was not obtained",
    "insurer_name": "Keystone Health Plan",
    "patient_name": "Jane Smith",
    "date_of_service": "01/15/2026",
    "date_of_denial": "02/01/2026",
    "claim_number": "CLM-98765",
    "plan_type": "aca",
    "confidence": "high"
  },
  "denial_category": "prior_authorization",
  "track_info": {
    "regulations": [{"citation": "...", "summary": "..."}],
    "appeal_deadline": "30 days (internal), 4 months (external)",
    "appeal_strategy": "...",
    "escalation": "...",
    "required_evidence": ["..."]
  },
  "appeal_letter": "Dear Appeals Department,\n\nI am writing to formally appeal..."
}
```

Response (need plan type):

```json
{
  "status": "need_plan_type",
  "extracted_fields": { ... },
  "denial_category": "prior_authorization",
  "message": "We identified your denial type but need to know your insurance type."
}
```

Response (low confidence):

```json
{
  "status": "low_confidence",
  "message": "We think this might be a prior_authorization denial, but we're not con
  "tentative_category": "prior_authorization",
  "suggestion": "Please verify: does this look right?"
}
```

Response (failure):

```json
{
  "status": "insufficient_data",
  "message": "We couldn't identify the basis for this denial.",
  "suggestion": "Try entering your denial code manually, or contact your state insur
}
```

POST `/api/analyze/manual`

Request: form data with fields as described in backend section

Response: Same structure as document endpoint

GET `/api/categories`

Response:

```json
[
  {"id": "prior_authorization", "name": "Prior Authorization", "codes": ["197", "15"
  {"id": "medical_necessity", "name": "Medical Necessity", "codes": ["50", "150", "1
  ...
]
```

DEMO SCRIPT (3 minutes)

Slide 1 (20 sec): "Over half of insurance denials are overturned on appeal, but most people never appeal because they don't know how. Our tool covers all 4 major insurance types — ACA, Medicare, employer plans, and Medicaid — reaching virtually every insured American."

Demo 1 (40 sec): Upload ACA prior auth denial PDF. Show extraction → confirmation → appeal letter with 42 USC §300gg-19 citation. "Every citation comes from our verified database."

Demo 2 (40 sec): Upload Medicare medical necessity denial. Show different appeal track — 5-level Medicare process, different deadlines, different regulations. "Same tool, completely different legal strategy."

Demo 3 (30 sec): Upload Medicaid coverage denial. Show federal fair hearing rights plus Pennsylvania-specific agency contact info auto-populated. "We cover all 50 states' Medicaid offices."

Demo 4 (20 sec): Quick manual entry — select employer plan, type CO-16. Get ERISA appeal letter with 29 USC §1133 citation and warning about building evidentiary record. "Works even without a document."

Close (30 sec): "We don't promise to win your appeal. We promise that if you have grounds to appeal, you won't lose just because you didn't know how to ask."